```
using CityBreaks.AuthorizationRequirements;
using Microsoft.AspNetCore.Authorization;

namespace CityBreaks.AuthorizationHandlers
{
    public class HasClaimHandler : AuthorizationHandler<ViewRolesRequirement>
    {
        protected override Task HandleRequirementAsync(AuthorizationHandlerContext
context,
            ViewRolesRequirement req)
        {
            var joiningDateClaim =
            ➥ context.User.FindFirst(c => c.Type == "Joining Date")?.Value;
            if (joiningDateClaim == null)
            {
                return Task.CompletedTask;
            }
            var joiningDate = Convert.ToDateTime(joiningDateClaim);

            if (context.User.HasClaim("Permission", "View Roles") &&
                  joiningDate < DateTime.Now.AddMonths(req.Months))
            {
                context.Succeed(req);
            }
            return Task.CompletedTask;
        }
    }
}
```

Next you add a new class named `IsInRoleHandler`, which is responsible for handling the additional condition for authorization. The code is very similar to the previous handler.

Listing 10.24 The `IsInRoleHandler`

```
using CityBreaks.AuthorizationRequirements;
using Microsoft.AspNetCore.Authorization;

namespace CityBreaks.AuthorizationHandlers
{
    public class IsInRoleHandler : AuthorizationHandler<ViewRolesRequirement>
    {
        protected override Task
        ➥ HandleRequirementAsync(AuthorizationHandlerContext context,
            ViewRolesRequirement req)
        {
            if (context.User.IsInRole("Admin"))
            {
```